

Reference_RBC_simulation

May 15, 2024

```
[ ]: """
This notebook presents guides for simulating 2D horizontally-periodic
Rayleigh-Benard convection
using Dedalus framework as written in following site
https://dedalus-project.readthedocs.io/en/latest/pages/examples/
ivp_2d_rayleigh_benard.html#
"""

import numpy as np
import matplotlib.pyplot as plt
import dedalus.public as d3
```

Computational domain of problem is $[L_x \times L_z] := [4 \times 1]$ with a corresponding mesh of $[N_x \times N_z] := [256 \times 64]$

```
[ ]: # Parameters
Xmin, Xmax = 0., 1.      # computational domain of x-dimension
Zmin, Zmax = 0., 1.      # computational domain of z-dimension
Nx, Nz = 64, 64          # number of points
```

The system is governed by

$$\nabla \cdot \vec{u} = 0, \quad (1)$$

$$\partial_t \vec{u} + \vec{u} \cdot \nabla \vec{u} = \nu \nabla^2 \vec{u} - \nabla p + B \vec{e}_z, \quad (2)$$

$$\partial_t B + \vec{u} \cdot \nabla B = \kappa \nabla^2 B. \quad (3)$$

where $\vec{u}$, p , B , $\vec{e}_z$, κ , and ν are the fluid velocity, pressure, buoyancy term, unit vector in the vertical, thermal diffusivity, and kinematic viscosity. The buoyancy term is used for representing the temperature and to simplify computation.

In this case, we non-dimensionalize the problem by using domain height $h = L_z = 1$ and the freefall time. Therefore, the Rayleigh number and Prandtl number can be determined by

$$Ra := \frac{1}{\nu\kappa}, \quad Pr := \frac{\nu}{\kappa} \quad (4)$$

Based on that, we can obtain the thermal diffusivity κ and kinematic viscosity ν .

$$\Rightarrow \quad \kappa := (RaPr)^{-1/2}, \quad \nu := \left(\frac{Ra}{Pr}\right)^{-1/2} \quad (5)$$

```
[ ]: Ra = 2e6      # Rayleigh number
     Pr = 1.       # Prandtl number
     kappa = (Ra*Pr)**(-1./2.) # thermal diffusivity
     nu = (Ra/Pr)**(-1./2.)    # kinematic viscosity
```

Governing equations are modified to fit with the Dedalus framework:

$$\nabla \cdot \vec{u} = 0, \quad (6)$$

$$\partial_t \vec{u} - \nu \nabla^2 \vec{u} + \nabla p - B \vec{e}_z = -\vec{u} \cdot \nabla \vec{u}, \quad (7)$$

$$\partial_t B - \kappa \nabla^2 B = -\vec{u} \cdot \nabla B. \quad (8)$$

To begin with, we must create basis for problem, including the mesh system for present 2D problem and defining data type of output.

```
[ ]: dtype = np.float64 # = double
     dealias = 3/2

     # Bases
     coords = d3.CartesianCoordinates('x','z')
     dist = d3.Distributor(coords, dtype=dtype)
     xbasis = d3.RealFourier(coords['x'], size=Nx, bounds=(Xmin, Xmax),
                               ↪ dealias=dealias)
     zbasis = d3.ChebyshevT(coords['z'], size=Nz, bounds=(Zmin, Zmax),
                              ↪ dealias=dealias)
```

Now, we will instantiate fields which will appears within the problem, including velocity $\vec{u}$, buoyancy b and pressure p .

```
[ ]: # Fields
     p = dist.Field(name='p', bases=(xbasis,zbasis)) # create pressure field
     b = dist.Field(name='b', bases=(xbasis,zbasis)) # create buoyancy field
     u = dist.VectorField(coords, name='u', bases=(xbasis,zbasis)) # create velocity
                               ↪ field
```

First, we consider the incompressible flow condition's equation:

$$\nabla \cdot \vec{u} = 0 \quad (9)$$

When implementing this divergence equation, there is a few error if one variable is underdetermined. So, we will use Gauge condition in present Dedalus v3, read here [https://dedalus-project.readthedocs.io/en/latest/pages/gauge_conditions.html]. Here, the divergence equation is expanded the system by adding a spatially-constant variable (τ_p) for solving pressure field. As a result, the equation is replaced by

$$\nabla \cdot \vec{u} + \tau_p = 0 \quad (10)$$

In Dedalus, we need to create a corresponding sub-field for τ_p

```
[ ]: # Substitutions
x, z = dist.local_grids(xbasis, zbasis)
ex, ez = coords.unit_vector_fields(dist)
lift_basis = zbasis.derivative_basis(1)
lift = lambda A: d3.Lift(A, lift_basis, -1)

# create constant sub-field for incompressible flow condition's equation
tau_p = dist.Field(name='tau_p')
# because this term is only a constant added to the equation, we don't need to
↳ instantiate it for bases system

# create constant sub-field for bouyancy term
tau_b1 = dist.Field(name='tau_b1', bases=xbasis)
tau_b2 = dist.Field(name='tau_b2', bases=xbasis)
grad_b = d3.grad(b) + ez*lift(tau_b1) # First-order reduction

tau_u1 = dist.VectorField(coords, name='tau_u1', bases=xbasis)
tau_u2 = dist.VectorField(coords, name='tau_u2', bases=xbasis)
grad_u = d3.grad(u) + ez*lift(tau_u1) # First-order reduction
```

When we have all the fields for solving, we must build the IVP problem and introduce fields to the solver. At this step, we don't have full field for solving all three governing equations, so we just introduce corresponding codes to perform this.

Build problem for the solver

```
[ ]: # Problem
problem = d3.IVP([p, tau_p, u, tau_u1, tau_u2, b, tau_b1, tau_b2],
↳ namespace=locals())
```

Adding the first equation

$$\text{tr}(\nabla \cdot \vec{u}) + \tau_p = 0 \quad (11)$$

```
[ ]: # equation 1
problem.add_equation("trace(grad_u) + tau_p = 0")
problem.add_equation("integ(p) = 0") # Pressure gauge
```

```
[ ]: {'LHS': Integrate(Integrate(<Field 140621708675408>)),
      'RHS': 0,
      'condition': 'True',
      'tensorsig': (),
      'dtype': numpy.float64,
      'M': 0,
      'L': Integrate(Integrate(<Field 140621708675408>)),
      'F': <Field 140621707752848>,
      'domain': <dedalus.core.domain.Domain at 0x7fe50afd5730>,
      'matrix_dependence': array([ True,  True]),
      'matrix_coupling': array([False,  True])}
```

Adding the second equation

$$\partial_t \vec{u} - \nu \nabla^2 \vec{u} + \nabla p - B \vec{e}_z = -\vec{u} \cdot \nabla \vec{u} \quad (12)$$

```
[ ]: # equation 2
problem.add_equation("dt(u) - nu*div(grad_u) + grad(p) - b*ez + lift(tau_u2) =_
    ↪ u@grad(u)")
```

```
[ ]: {'LHS': Add(Add(Add(Add(Convert(dt(<Field 140621707835632>)), Mul(-1,
Mul(0.0007071067811865475, Div(Add(Grad(<Field 140621707835632>), Mul(<Field
140621708622448>, Mul(<Field 140621707808928>, <Field 140621707707888>)))))),
Convert(Grad(<Field 140621708675408>))), Convert(Mul(-1, Mul(<Field
140621753848160>, <Field 140621708622448>)))), Convert(Mul(<Field
140621707808928>, <Field 140621710065792>))),
      'RHS': Dot(Mul(-1, <Field 140621707835632>), Grad(<Field 140621707835632>)),
      'condition': 'True',
      'tensorsig': (<dedalus.core.coords.CartesianCoordinates at 0x7fe50af06b40>,),
      'dtype': dtype('float64'),
      'M': Convert(<Field 140621707835632>),
      'L': Add(Add(Add(Mul(-1, Mul(0.0007071067811865475, Div(Add(Grad(<Field
140621707835632>), Mul(<Field 140621708622448>, Mul(<Field 140621707808928>,
<Field 140621707707888>)))))), Convert(Grad(<Field 140621708675408>))),
Convert(Mul(-1, Mul(<Field 140621753848160>, <Field 140621708622448>)))),
Convert(Mul(<Field 140621707808928>, <Field 140621710065792>))),
      'F': Convert(Dot(Mul(-1, <Field 140621707835632>), Grad(<Field
140621707835632>))))),
```

```
'domain': <dedalus.core.domain.Domain at 0x7fe50af24c20>,
'matrix_dependence': array([ True,  True]),
'matrix_coupling': array([False,  True])}
```

Adding the third equation

$$\partial_t B - \kappa \nabla^2 B = -\vec{u} \cdot \nabla B \quad (13)$$

```
[ ]: # equation 3
problem.add_equation("dt(b) - kappa*div(grad_b) + lift(tau_b2) = - u@grad(b)")
```

```
[ ]: {'LHS': Add(Add(Convert(dt(<Field 140621753848160>)), Mul(-1,
Mul(0.0007071067811865475, Div(Add(Grad(<Field 140621753848160>), Mul(<Field
140621708622448>, Mul(<Field 140621707808928>, <Field 140621708636336>))))))),
Convert(Mul(<Field 140621707808928>, <Field 140621707837408>))),
'RHS': Dot(Mul(-1, <Field 140621707835632>), Grad(<Field 140621753848160>)),
'condition': 'True',
'tensorsig': (),
'dtype': dtype('float64'),
'M': Convert(<Field 140621753848160>),
'L': Add(Mul(-1, Mul(0.0007071067811865475, Div(Add(Grad(<Field
140621753848160>), Mul(<Field 140621708622448>, Mul(<Field 140621707808928>,
<Field 140621708636336>))))))), Convert(Mul(<Field 140621707808928>, <Field
140621707837408>))),
'F': Convert(Dot(Mul(-1, <Field 140621707835632>), Grad(<Field
140621753848160>))),
'domain': <dedalus.core.domain.Domain at 0x7fe50af24c20>,
'matrix_dependence': array([ True,  True]),
'matrix_coupling': array([False,  True])}
```

Set up the boundary conditions

```
[ ]: # for buoyancy
problem.add_equation("b(z=Zmin) = 1")
problem.add_equation("b(z=Zmax) = 0")
# for velocity
problem.add_equation("u(z=Zmin) = 0")
problem.add_equation("u(z=Zmax) = 0")
```

```
[ ]: {'LHS': interp(<Field 140621707835632>, z=1.0),
'RHS': 0,
'condition': 'True',
'tensorsig': (<dedalus.core.coords.CartesianCoordinates at 0x7fe50af06b40>,),
'dtype': numpy.float64,
'M': 0,
'L': interp(<Field 140621707835632>, z=1.0),
'F': <Field 140621752173104>,
```

```
'domain': <dedalus.core.domain.Domain at 0x7fe50af22570>,
'matrix_dependence': array([False,  True]),
'matrix_coupling': array([False,  True])}
```

Solver

```
[ ]: stop_sim_time = 50
      timestepper = d3.RK222
      # Solver
      solver = problem.build_solver(timestepper)
      solver.stop_sim_time = stop_sim_time
```

```
2024-05-15 13:57:27,341 subsystems 0/1 INFO :: Building subproblem matrices 1/32
(~3%) Elapsed: 0s, Remaining: 3s, Rate: 1.0e+01/s
2024-05-15 13:57:27,543 subsystems 0/1 INFO :: Building subproblem matrices 4/32
(~12%) Elapsed: 0s, Remaining: 2s, Rate: 1.3e+01/s
2024-05-15 13:57:27,829 subsystems 0/1 INFO :: Building subproblem matrices 8/32
(~25%) Elapsed: 1s, Remaining: 2s, Rate: 1.4e+01/s
2024-05-15 13:57:28,097 subsystems 0/1 INFO :: Building subproblem matrices
12/32 (~38%) Elapsed: 1s, Remaining: 1s, Rate: 1.4e+01/s
2024-05-15 13:57:28,409 subsystems 0/1 INFO :: Building subproblem matrices
16/32 (~50%) Elapsed: 1s, Remaining: 1s, Rate: 1.4e+01/s
2024-05-15 13:57:28,753 subsystems 0/1 INFO :: Building subproblem matrices
20/32 (~62%) Elapsed: 2s, Remaining: 1s, Rate: 1.3e+01/s
2024-05-15 13:57:29,128 subsystems 0/1 INFO :: Building subproblem matrices
24/32 (~75%) Elapsed: 2s, Remaining: 1s, Rate: 1.3e+01/s
2024-05-15 13:57:29,398 subsystems 0/1 INFO :: Building subproblem matrices
28/32 (~88%) Elapsed: 2s, Remaining: 0s, Rate: 1.3e+01/s
2024-05-15 13:57:29,672 subsystems 0/1 INFO :: Building subproblem matrices
32/32 (~100%) Elapsed: 2s, Remaining: 0s, Rate: 1.3e+01/s
```

Set up initial conditions

```
[ ]: b.fill_random('g', seed=42, distribution='normal', scale=1e-3) # Random noise
      b['g'] *= z * (Zmax - z) # Damp noise at walls
      b['g'] += Zmax - z # Add linear background
```

```
[ ]: # Analysis
      snapshots = solver.evaluator.add_file_handler('snapshots', sim_dt=0.25,
      ↪max_writes=50)
      snapshots.add_task(b, name='buoyancy')
      snapshots.add_task(-d3.div(d3.skew(u)), name='vorticity')
```

```
[ ]: max_timestep = 0.125
      # CFL
      CFL = d3.CFL(solver, initial_dt=max_timestep, cadence=10, safety=0.5,
      ↪threshold=0.05,
      max_change=1.5, min_change=0.5, max_dt=max_timestep)
      CFL.add_velocity(u)
```

```
[ ]: # Flow properties
flow = d3.GlobalFlowProperty(solver, cadence=10)
flow.add_property(np.sqrt(u@u)/nu, name='Re')

[ ]: from dedalus.extras.plot_tools import plot_bot_2d

figkw = {'figsize':(1,1), 'dpi':200}
# Main loop
# timestep = 0.05
print('Starting main loop')
while solver.proceed:
    timestep = CFL.compute_timestep()
    solver.step(timestep)
    if solver.iteration % 1000 == 0:
        print('Completed iteration {}, time={:.3f}'.format(solver.iteration,
        ↪ solver.sim_time))
        print(b['g'])
        b.layout.grid_space
        plot_bot_2d(b, figkw=figkw, title="b['g']")
        plt.savefig("Reference_RBC_simulation_bouyancy_t{:.3f}.png".
        ↪ format(solver.sim_time), dpi=200)
        # plt.close()
```

Starting main loop

Completed iteration 1000, time=23.962

```
[9.99055666e-01 9.91502375e-01 9.76401873e-01 ... 2.63877141e-02
 9.50352520e-03 1.05614499e-03]
[9.99008682e-01 9.91079539e-01 9.75227621e-01 ... 2.55457725e-02
 9.20026096e-03 1.02244701e-03]
[9.98961754e-01 9.90657213e-01 9.74054829e-01 ... 2.46840320e-02
 8.88986374e-03 9.87955786e-04]
```

...

```
[9.99195026e-01 9.92756521e-01 9.79884807e-01 ... 2.87928685e-02
 1.03698597e-02 1.15241296e-03]
[9.99149018e-01 9.92342487e-01 9.78734983e-01 ... 2.80110914e-02
 1.00882639e-02 1.12112090e-03]
[9.99102506e-01 9.91923916e-01 9.77572550e-01 ... 2.72094743e-02
 9.79952146e-03 1.08903561e-03]]
```

Completed iteration 2000, time=37.147

```
[9.99842206e-01 9.98511923e-01 9.95720579e-01 ... 3.88661167e-03
 1.45335489e-03 1.70309229e-04]
[9.99849897e-01 9.98579499e-01 9.95907338e-01 ... 3.83223700e-03
 1.43420312e-03 1.68029773e-04]
[9.99853113e-01 9.98608533e-01 9.95989857e-01 ... 3.87950091e-03
 1.45045406e-03 1.69526539e-04]
```

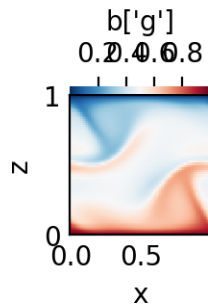
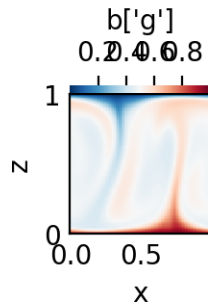
...

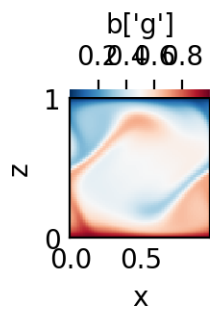
```
[9.99807867e-01 9.98214476e-01 9.94906453e-01 ... 4.65445848e-03
 1.72479294e-03 2.00468000e-04]
```

```

[9.99819878e-01 9.98318332e-01 9.95190799e-01 ... 4.30950575e-03
 1.60229303e-03 1.86845399e-04]
[9.99831703e-01 9.98420574e-01 9.95470144e-01 ... 4.04784831e-03
 1.51001005e-03 1.76629180e-04]]
Completed iteration 3000, time=49.766
[[9.99627555e-01 9.96985313e-01 9.92370722e-01 ... 5.54516921e-03
 1.72996812e-03 1.48609454e-04]
[9.99708766e-01 9.97366900e-01 9.92757090e-01 ... 5.88568036e-03
 2.09431986e-03 2.23803659e-04]
[9.99796025e-01 9.97788120e-01 9.93165300e-01 ... 6.23310482e-03
 2.49419668e-03 3.11119017e-04]
...
[9.99666694e-01 9.96947994e-01 9.91358061e-01 ... 4.81045033e-03
 1.79591631e-03 2.12502135e-04]
[9.99620188e-01 9.96842246e-01 9.91701335e-01 ... 5.01392389e-03
 1.63944998e-03 1.60142131e-04]
[9.99595864e-01 9.96806887e-01 9.92025529e-01 ... 5.25040309e-03
 1.57119263e-03 1.26589946e-04]]
2024-05-15 13:58:32,586 solvers 0/1 INFO :: Simulation stop time reached.

```





[]: